Assignment #4 - Facade Pattern

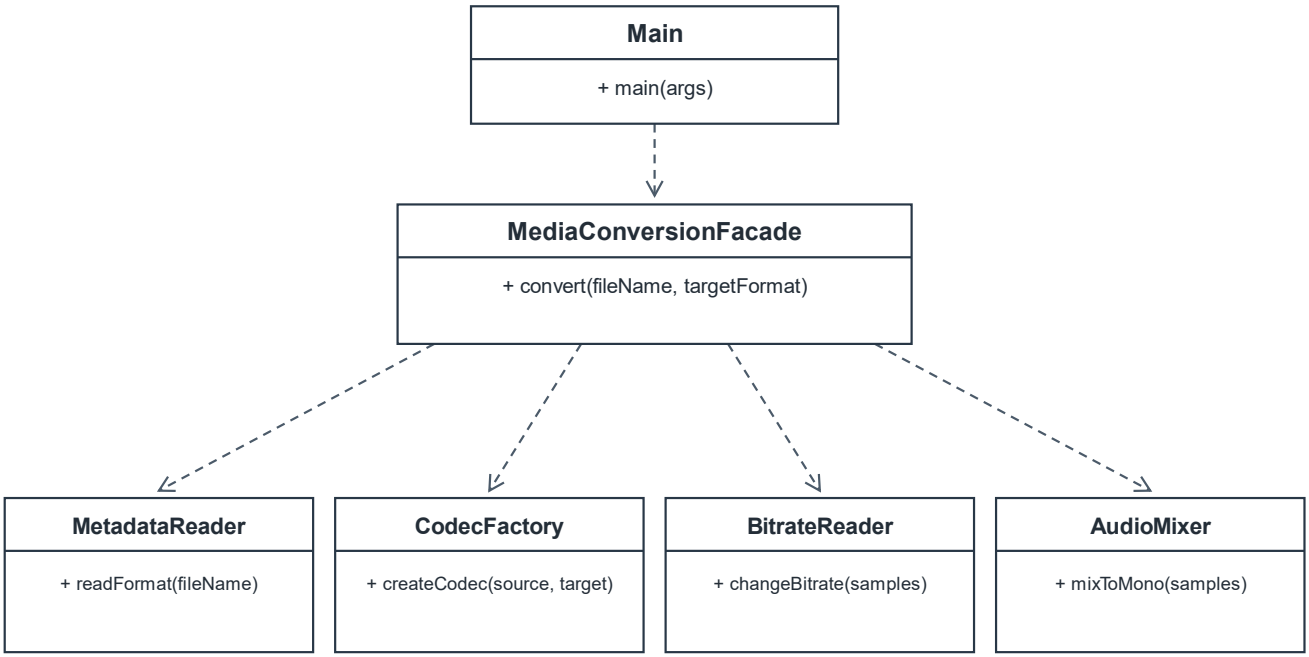
Student: Arnur Kozhabergen
Group: se-2529
Topic: Media/File Conversion Facade

1. Introduction

Media conversion has several steps: read the source format, choose a codec, adjust sample data, and combine stereo channels. MediaConversionFacade exposes one convert(fileName, targetFormat) method and runs these steps in order.

The example uses simulated audio samples. It demonstrates the conversion flow without reading or writing real media files.

2. UML Class Diagram



Dashed arrows show dependency: Main uses the Facade, and the Facade coordinates the four independent subsystems.

3. Clean Code Principles

3.1 Meaningful names

```
Before: run(a, b)
After:  convert(fileName, targetFormat)
```

The method and parameter names tell the reader what the operation does and what values it needs.

3.2 Single Responsibility

```
Before: One method reads metadata, selects a codec, changes bitrate, and mixes channels.
After:  MetadataReader, CodecFactory, BitrateReader, AudioMixer
```

Each subsystem handles one conversion task; the Facade only coordinates them.

3.3 Hide subsystem details from the client

```
Before: Main calls readFormat(), createCodec(), changeBitrate(), and mixToMono().
After:  facade.convert("song.mp3", "wav")
```

Main uses one Facade method instead of managing the conversion steps itself.

3.4 Keep classes focused

```
Before: class MediaConverter { format; codec; bitrate; mixer; }
After:  MetadataReader.readFormat(); AudioMixer.mixToMono();
```

The classes contain only the work related to their names. CodecFactory and BitrateReader follow the same structure.

3.5 Report errors clearly

```
Before: Unsupported input fails later with no useful message.
After:  throw new IllegalArgumentException("Only mp3 and wav formats are supported");
```

CodecFactory rejects unsupported formats, and Main catches the exception and prints its message.

4. Conclusion

The Facade hides the order of subsystem calls while keeping those public classes available for direct use. This demo processes sample arrays and prints results; it does not create an encoded media file.